Client Cybersecurity TPRA Agentic MVP — Complete Deep Dive Guide

Document purpose: This is a NotebookLM-ready master source. Upload this file into NotebookLM to ask questions, generate study guides, briefings, FAQs, and audio overviews about the project.

Project path: `Dev/tpra-agentic-mvp` (also linked from `/Dev/tpra-agentic-mvp`)
Audience: Engineers, architects, reviewers, and stakeholders who need a clear and deep understanding of the system.
Status: Agentic MVP scaffold with working local mock providers and end-to-end UC1 → UC2 pipeline.

---

1. Executive Summary

This project is a **human-in-the-loop agentic platform** for a client's **Third Party Risk Assessment (TPRA)** process.

In plain language:

- Security teams receive raw vendor security findings in messy formats (Excel, CSV, JSON, Word).
- Those findings must be cleaned, standardized, validated, reviewed by humans, and then turned into a formal TPRA report.
- This platform uses two AI agents to accelerate that work while keeping humans in control of approvals.

The two agents are:

1. **UC1 — Structured Findings Package Agent**
   Takes raw findings → normalizes them → validates them → creates a structured package and exception report.

2. **UC2 — Draft TPRA Report Generation Agent**
   Takes approved findings → maps them into report sections → uses an LLM to draft narrative text → produces a Word (DOCX) draft for human review.

The core design philosophy is **"build once, run anywhere."**
The same application code can run:

- **Locally** for Development Team work (filesystem, SQLite, mock LLM, header-based auth), or
- **On Azure / Client DaVinci** (Blob Storage, Cosmos DB, Azure OpenAI / AI Foundry, Client SSO).

Infrastructure differences are hidden behind a **Provider Registry**. Changing environments is primarily a configuration change, not a rewrite.

---

2. Why This Project Exists (Business Context)

2.1 What is TPRA?

Third Party Risk Assessment (TPRA) is the process of evaluating the cybersecurity posture of vendors, suppliers, and other external parties. Organizations use TPRA to decide whether a vendor is safe enough to do business with, what risks remain, and what remediation is required.

Typical TPRA inputs include:

- Vendor questionnaires
- Security assessment findings
- Pen-test or scanning outputs
- Policy/control evidence
- Exception lists and open issues

Typical TPRA outputs include:

- A cleaned, structured findings inventory
- Exception / gap tracking
- Reviewer decisions (approve / reject / escalate)

- A narrative assessment report for risk committees, auditors, or client stakeholders

2.2 Pain points this MVP addresses

Without automation, TPRA teams often struggle with:

- Inconsistent finding formats from different vendors
- Manual copy/paste into report templates
- Missing required fields that are discovered late
- Weak audit trails of who approved what
- Slow draft report writing

This MVP targets the highest-friction steps:

1. Normalize and validate findings (UC1)
2. Human review checkpoint
3. Draft the report narrative and Word document (UC2)

2.3 Human-in-the-loop principle

This is intentionally **not** a fully autonomous black box.

- Agents prepare structured work products.
- Humans review findings between UC1 and UC2.
- Humans remain accountable for final report quality.
- Every important action is written to an audit trail.

That balance is critical for regulated cybersecurity workflows.

---

3. Product Scope of the MVP

In scope

- Workspace management for one TPRA engagement at a time
- File upload/download for findings and generated outputs
- UC1 agent execution
- Reviewer decisions on findings
- UC2 draft report generation
- Audit event history

- Local development mode with mock providers
- Azure-ready provider interfaces and deployment scaffolding

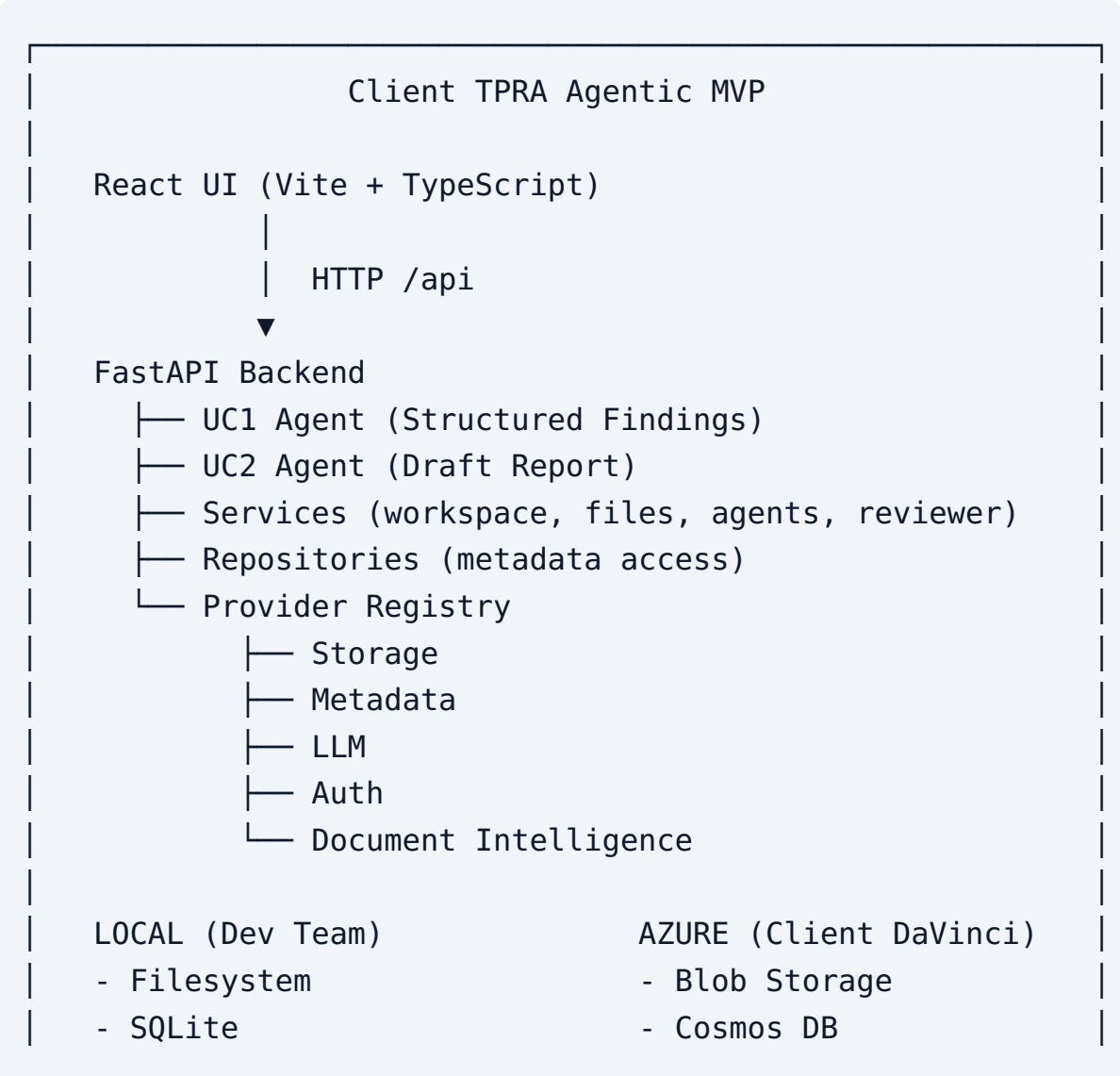
Out of scope / future enhancements (typical)

- Full production SSO hardening and entitlement sync
- Complex multi-approver workflow engines
- Advanced RAG over historical TPRA corpora
- Fully automated remediation tracking
- Multi-tenant SaaS packaging beyond client deployment

---

4. High-Level System Architecture

4.1 Architecture diagram (conceptual)



```
│   - Mock LLM                      - OpenAI / Foundry      │
│   - Dev Auth headers             - Client SSO            │
└──────────────────────────────────────────────────────────┘
```

4.2 Clean architecture layers

The backend is organized so business logic does not depend on Azure SDKs directly.

Layer	Folder	Responsibility
API	`backend/app/api`	HTTP routes, request validation, dependency injection
Schemas	`backend/app/schemas`	Pydantic request/response contracts
Services	`backend/app/services`	Orchestration, authorization, business workflows
Agents	`backend/app/agents`	UC1/UC2 step graphs and domain-specific agent logic
Domain	`backend/app/domain`	Canonical models, enums, validation rules
Repositories	`backend/app/repositories`	Persist/retrieve domain objects via metadata provider
Providers	`backend/app/providers`	Infrastructure adapters (local + Azure)
Core	`backend/app/core`	Config, logging, exceptions

4.3 Why this layering matters

- You can unit-test normalizers/validators without Azure.
- You can swap SQLite for Cosmos without rewriting agents.
- You can run demos offline with a mock LLM.
- API contracts stay stable while internals evolve.

---

5. End-to-End Business Flow

This is the most important mental model for the system.

Step A — Create a workspace

A **workspace** represents one TPRA engagement (for example, “Vendor ACME — Q3 assessment”).

All files, agent runs, reviewer decisions, and audit events are scoped to that workspace.

Step B — Upload raw findings

An analyst uploads one or more source files:

- `.xlsx` / `.xls`
- `.csv`
- `.json`
- `.docx`

These are stored through the Storage provider. Metadata about each file is stored through the Metadata provider.

Step C — Run UC1

UC1 reads the uploaded files and produces:

- Structured findings package (normalized inventory)
- Exception report (missing/invalid fields)
- Reviewer log template / reviewer-ready output

Step D — Human review

Reviewers inspect findings and record decisions:

- Approve
- Reject
- Optionally comment

Only authorized roles (reviewer/admin) can record decisions.

Step E — Run UC2

UC2 consumes the approved findings package and produces:

- Draft TPRA Word report (`.docx`)
- Missing-input report (if needed)
- Summary artifacts for review

Step F — Human finalization

A human edits/approves the Word draft outside or inside the platform workflow. The system's job in the MVP is to produce a high-quality first draft quickly and transparently.

Step G — Auditability

Every meaningful action (workspace create, file upload, agent start/success/fail, reviewer decision) is recorded as an immutable audit event.

---

6. Agent Deep Dive — UC1 Structured Findings Package Agent

6.1 Purpose

UC1 converts inconsistent vendor findings into a **canonical TPRA schema**.

6.2 Agent identity

- **ID:** `uc1_structured_findings`
- **Name:** Structured Findings Package Agent
- **Required input:** at least one findings file
- **Prompt directory:** `prompts/structured_findings/v1`
- **Registry source:** `foundry/agents.yaml`

6.3 UC1 workflow steps

UC1 is implemented as a sequential workflow graph. Conceptually:

1. **Validate inputs** — ensure files exist

2. **Authorize** — user must have `agent:run`
3. **Load files** — pull bytes + metadata into state
4. **Detect types** — csv / json / xlsx / docx / unknown
5. **Extract content** — parser converts each file into raw row dictionaries
6. **Normalize** — map vendor field names into canonical Finding fields
7. **Validate fields** — apply business rules; mark exceptions
8. **Capture reviewer log readiness**
9. **Build package outputs** — structured package + exception report + reviewer log
10. **Audit + respond**

If extraction finds zero rows, the workflow can short-circuit as failed.

6.4 Parsing strategy

Parsers live in `backend/app/agents/structured_findings/parsers.py`.

Format	Behavior
JSON	Accepts list, or object with `findings`, or single object
CSV	Uses header row as field names
XLSX	Uses first sheet, first row as headers
DOCX	Extracts paragraph text into a document-style finding

This makes UC1 resilient to common vendor export formats.

6.5 Normalization strategy

Normalizer maps many possible column names to canonical fields:

- `title` aliases: title, finding, name, summary
- `description` aliases: description, details, detail, narrative
- `severity` aliases: severity, risk, priority, level
- `category` aliases: category, domain, control_area, type
- `source` aliases: source, vendor, origin

Each normalized item becomes a `Finding` domain object with a generated ID.

6.6 Validation rules

Validation rules live in domain logic and check things like:

- Required fields present (`title`, `severity`, `category`)
- Severity values belong to an allowed set (critical/high/medium/low/info)

Invalid findings are not silently dropped. They are flagged as exceptions so humans can remediate.

6.7 UC1 outputs

Typical outputs:

- Structured findings package (CSV/XLSX)
- Exception report (JSON or spreadsheet-style)
- Reviewer log sheet for decision capture

These outputs are stored as files in the workspace and referenced by the agent run record.

6.8 Why UC1 is valuable even before generative AI

UC1 is mostly deterministic data engineering + validation. That is intentional:

- High reliability
- Easy testing
- Clear auditability
- Strong foundation before LLM drafting in UC2

---

7. Agent Deep Dive — UC2 Draft TPRA Report Generation Agent

7.1 Purpose

UC2 turns approved structured findings into a readable draft TPRA report.

7.2 Agent identity

- **ID:** `uc2_draft_report`

- **Name:** Draft TPRA Report Generation Agent
- **Required input:** approved findings package
- **Prompt directory:** `prompts/draft_report/v1`

7.3 UC2 workflow steps

Conceptually:

1. Validate + authorize
2. Load approved findings package
3. Check approvals / usable findings
4. Load report template mapping rules
5. Map findings into report sections by category
6. Invoke LLM with system + user prompts to draft narrative
7. Detect missing inputs
8. Build DOCX report
9. Emit missing-input report + summary
10. Audit + respond

7.4 Template mapping

`template_mapper` groups findings by category (for example identity, cloud, logging).
Each category becomes a report section heading with related bullets/findings.

7.5 LLM role in UC2

The LLM is used for **narrative drafting**, not for inventing findings.

Prompt guidance emphasizes:

- Audit-ready language
- Clear distinction between facts and recommendations
- Grouping by control category
- Staying faithful to provided findings

In local development, a **Mock LLM** returns deterministic draft text so the pipeline works offline.

In client cloud mode, Azure OpenAI or Azure AI Foundry provides real generation.

7.6 DOCX generation

`report_builder` creates a Word document containing:

- Report title
- LLM narrative
- Section headings
- Finding bullets with severity and description

This is the artifact reviewers expect in traditional TPRA processes.

---

8. Workflow Engine Internals

File: `backend/app/agents/graph.py`

The engine is intentionally simple:

- An ordered list of named step functions
- Shared mutable `state` dictionary
- Per-step timing/status traces
- `StopWorkflow` exception for controlled early exit

Why a custom lightweight graph instead of a heavy orchestration framework?

- MVP clarity
- Easy debugging
- Deterministic step traces for audits/demos
- Enough structure to later migrate to LangGraph/Foundry orchestration if needed

Each run can record step traces such as:

- step name
- status (`ok`, `stopped`, `error`)
- duration
- error message if any

---

9. Domain Model Deep Dive

Domain models are the “nouns” of the system. Every layer should speak this language.

9.1 Core entities

Workspace

Represents one assessment engagement.

Important fields:

- `id`
- `name`
- `description`
- timestamps

FileRecord

Metadata about an uploaded or generated file.

Important fields:

- `id`
- `workspace_id`
- `filename`
- `content_type`
- `storage_key`
- `size_bytes`

Finding

Canonical security finding object.

Important fields:

- `id`
- `title`
- `description`
- `severity`

- `category`
- `source`
- `status` (new/valid/exception/approved/rejected)
- `raw` original row payload

ExceptionItem

Validation issue tied to a finding/field.

Run

One execution of an agent.

Important fields:

- `id`
- `workspace_id`
- `agent_type`
- `status` (pending/running/succeeded/failed)
- `input_file_ids`
- `output_refs`
- `error`

ReviewerDecision

Human approval/rejection record for a finding.

AuditEvent

Append-only record of who did what, when, on which resource.

UserContext

Authenticated user identity + role used for authorization.

9.2 Enumerations

Enums prevent magic strings from spreading through code:

- `RunStatus`
- `FindingStatus`

- `ApprovalLevel`
- `UserRole` (analyst, reviewer, admin)
- `AgentType`
- Provider type selectors

9.3 Shared schema for prompts and agents

`prompts/shared/tpra_schema.json` defines the canonical Finding/Exception JSON schema used to keep LLM outputs and application models aligned.

---

10. Provider Registry — The Heart of Portability

10.1 Concept

The Provider Registry is a dependency-injection factory.
Services and agents ask for capabilities ("store this file", "generate text"), not for specific cloud products.

10.2 Provider interfaces

Defined in `backend/app/providers/base.py`:

1. **StorageProvider** — put/get/delete binary objects
2. **MetadataProvider** — upsert/get/list/delete JSON-like documents
3. **LLMProvider** — generate text from system/user prompts
4. **AuthProvider** — resolve current user context
5. **DocIntelligenceProvider** — extract text/structure from documents

10.3 Local vs Azure mapping

Capability	Local (Development Team)	Azure (Client DaVinci)
Storage	Local filesystem under `.data/`	Azure Blob Storage
Metadata	SQLite file DB	Cosmos DB
LLM	Mock deterministic responder	Azure OpenAI or AI Foundry

Capability	Local (Development Team)	Azure (Client DaVinci)
Auth	Trust `X-User-*` headers	Client SSO / OIDC JWT
Doc Intel	Local/basic parsers	Azure Document Intelligence

10.4 Configuration switch

Providers are selected from:

1. Environment variables (highest priority)
2. `backend/config.yaml`
3. Defaults

Examples:

- `STORAGE_PROVIDER=local|azure`
- `METADATA_PROVIDER=sqlite|cosmos`
- `LLM_PROVIDER=mock|azure_openai|foundry`
- `AUTH_PROVIDER=dev|client_sso`
- `DOC_INTEL_PROVIDER=local|azure`

There are ready presets:

- `.env.local.example` for local mock mode
- `.env.client.example` for client cloud mode
- `backend/config.client.example.yaml` for DaVinci-oriented config

10.5 Why stubs exist for Azure providers

Azure adapters may be stubbed or partially implemented in the MVP so local development stays lightweight. The interfaces and wiring are already in place, which is the important architectural commitment.

---

11. Services and Authorization

11.1 Service responsibilities

Service	Responsibility
`workspace_service`	Create/list/get/delete workspaces
`file_service`	Upload/list/download files via storage + metadata
`agent_service`	Create runs, invoke UC1/UC2, persist outputs
`reviewer_log_service`	Record and list reviewer decisions
`authz`	Role permission checks
Audit recording	Often embedded via audit repository/service calls

11.2 Role-based access control (RBAC)

Typical permission model:

Role	Can do
Analyst	Create workspaces, upload files, run agents, read data
Reviewer	Everything analyst can do + record approval decisions
Admin	Full access including delete and broad audit operations

Authorization failures raise structured auth errors rather than silently continuing.

11.3 Dev auth vs SSO

Dev auth makes local testing easy:

- Frontend sends headers like `X-User-Id`, `X-User-Name`, `X-User-Role`
- Backend trusts them in local mode

Client SSO is the production path:

- UI sends bearer JWT
- Backend validates issuer/audience/claims
- Role mapping comes from token claims or directory groups

Never use dev auth in production.

---

12. API Surface (How the UI and Integrations Talk to the Backend)

The API is FastAPI-based and self-documented at `/docs` when running locally.

12.1 Key endpoints

Area	Method / Path	Purpose
Health	`GET /api/health`	Liveness and provider summary
Catalog	`GET /api/catalog` (or `/api/v1/catalog`)	List available agents and required inputs
Workspaces	`GET/POST /api/workspaces`	List/create engagements
Workspace	`GET/DELETE /api/workspaces/{id}`	Fetch/delete one workspace
Files upload	`POST /api/files/upload`	Upload findings or supporting files
Files list	`GET /api/files/workspace/{id}`	List files in a workspace
Files download	`GET /api/files/{id}/download`	Download a stored file
Agent runs	`POST /api/agents/runs`	Start UC1 or UC2
Run status		Fetch one run

Area	Method / Path	Purpose
	`GET /api/agents/runs/{id}`	
Runs list	`GET /api/agents/workspace/{id}/runs`	List runs for workspace
Reviewer log	`POST /api/reviewer-log`	Record decision
Reviewer list	`GET /api/reviewer-log/workspace/{id}`	List decisions
Audit	`GET /api/audit/workspace/{id}`	Immutable activity history

Exact prefixing may be `/api` or `/api/v1` depending on router wiring; use OpenAPI docs as the live source of truth.

12.2 Correlation IDs

Each request can carry or receive an `x-correlation-id` header. Logs include this ID so a single user action can be traced across service boundaries.

12.3 Error shape

Custom exceptions (`TPRAError`, validation/auth/not-found variants) are converted into structured JSON errors with machine-readable codes.

---

13. Frontend Deep Dive

13.1 Stack

- React 18
- TypeScript
- Vite 5
- Simple fetch-based API client
- Nginx packaging for container deployment

13.2 Main UI capabilities (`frontend/src/App.tsx`)

The MVP UI focuses on operator workflow clarity:

1. Create/select a workspace
2. Upload findings files
3. Run UC1
4. Run UC2
5. Inspect run statuses and output references
6. View audit trail
7. Switch persona/role (analyst/reviewer/admin) for local testing

13.3 API client behavior

`frontend/src/api/client.ts`:

- Prefixes requests with configured API base URL
- Injects dev auth headers in local mode
- Can later inject SSO bearer tokens in client mode

13.4 Local proxying

Vite dev server proxies `/api` to `localhost:8000`, so the UI and API feel like one app during development.

In Docker/production UI image, Nginx serves the SPA and reverse-proxies `/api` to the backend service.

---

14. Prompts and Foundry Configuration

14.1 Why prompts are stored in Git

Prompts are first-class product assets:

- Versioned (`v1/`, future `v2/`)
- Reviewable in pull requests
- Deployable as an immutable Blob mirror
- Separated from application code so prompt iteration is controlled

14.2 Prompt layout

```
prompts/
  shared/tpra_schema.json
  structured_findings/v1/
    system.md
    user.md
    output_schema.json
  draft_report/v1/
    system.md
    user.md
```

14.3 Foundry registry

`foundry/agents.yaml` is the source of truth for:

- Agent IDs
- Names/descriptions
- Capabilities
- Required inputs
- Model designation
- Prompt directory references

Additional files:

- `foundry/structured-findings-agent.yaml`
- `foundry/draft-report-agent.yaml`

These support Azure AI Foundry deployment definitions (model, temperature, max tokens, tools).

---

15. Repository Map (File-by-File Understanding)

15.1 Root files

File	Why it exists
`.gitignore`	

File	Why it exists
	Prevent committing secrets, venv, node_modules, runtime data
`.env.example`	Documents all supported environment variables
`.env.local.example`	One-step local preset
`.env.client.example`	Client DaVinci preset
`docker-compose.yml`	One-command local full stack
`README.md`	Quickstart and architecture summary

15.2 Backend structure

```
backend/
  app/
    main.py
    core/
    domain/
    schemas/
    api/
    services/
    repositories/
    providers/
    agents/
  tests/
  config.yaml
  requirements.txt
  requirements-dev.txt
  Dockerfile
```

15.3 Frontend structure

```
frontend/
  src/
    main.tsx
```

```
    App.tsx
    api/client.ts
    config.ts
    styles.css
  index.html
  vite.config.ts
  package.json
  Dockerfile
  nginx.conf
```

15.4 Ops / delivery structure

```
scripts/                deploy helpers + prompt upload + foundry boots
infra/bicep/            Azure infrastructure as code
azure-pipelines/        multi-stage CI/CD
docs/                   architecture, migration, this NotebookLM guide
```

---

16. Infrastructure and CI/CD

16.1 Bicep infrastructure

`infra/bicep/main.bicep` provisions core Azure building blocks such as:

- Storage Account (TLS 1.2)
- Cosmos DB (session consistency)
- Key Vault (RBAC + soft delete)
- Log Analytics
- Application Insights

Stage parameter files:

- `dev.parameters.json`
- `qa.parameters.json`
- `uat.parameters.json`
- `prod.parameters.json`

16.2 Deployment scripts

- `scripts/_deploy_common.sh` shared routine
- `scripts/deploy_dev.sh`
- `scripts/deploy_qa.sh`
- `scripts/deploy_uat.sh`
- `scripts/deploy_prod.sh`
- `scripts/upload_prompts.py`
- `scripts/create_foundry_agents.py`

16.3 Azure Pipelines flow

Typical progressive delivery:

1. Test (pytest)
2. Build Docker images
3. Deploy DEV
4. Deploy QA
5. Deploy UAT (manual approval gates)

This reduces risk by promoting the same artifact style through environments.

---

17. Local Development Guide

17.1 Backend quickstart

```
cd Dev/tpra-agentic-mvp
cp .env.local.example .env
cd backend
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt
uvicorn app.main:app --reload --app-dir .
```

Open API docs: `http://localhost:8000/docs`

17.2 Frontend quickstart

```
cd frontend
npm install
npm run dev
```

Open UI: `http://localhost:5173`

17.3 Docker quickstart

```
cp .env.local.example .env
docker compose up --build
```

17.4 Tests

```
cd backend
source .venv/bin/activate
pytest
```

Expected coverage includes:

- Parser unit tests
- Normalizer unit tests
- Validator unit tests
- Agent registry loading tests
- UC1 → UC2 integration test with mock providers

17.5 Demo / smoke path

A smoke script or demo flow can:

1. Create workspace
2. Upload sample findings CSV/JSON
3. Run UC1
4. Run UC2 using UC1 package output
5. Download generated artifacts

Sample fixtures live under `backend/tests/fixtures/`.

---

18. Migration Path: Development Team Local → Client DaVinci

1. Copy `.env.client.example` to `.env` and fill real Azure/OIDC values.
2. Switch `backend/config.yaml` providers to Azure (or use `config.client.example.yaml`).
3. Provision Azure resources with Bicep.
4. Publish prompts to Blob using `upload_prompts.py`.
5. Create/update Foundry agents using `create_foundry_agents.py`.
6. Replace dev auth with Client SSO JWT validation.
7. Deploy through pipeline stages (DEV → QA → UAT → PROD).
8. Validate:
9. file upload/download via Blob
10. metadata persistence in Cosmos
11. real LLM draft quality
12. SSO login and role mapping
13. audit completeness

---

19. Security, Trust, and Compliance Considerations

19.1 Why audit events matter

TPRA evidence often supports risk acceptance and audit inquiries. The platform records:

- actor
- action
- resource type/id
- workspace
- timestamp
- details payload

19.2 Separation of duties

- Analysts prepare and run packages

- Reviewers approve findings
- Admins govern the environment

19.3 Data handling caution

Findings may contain sensitive security weaknesses. In production:

- Use private storage accounts
- Restrict Cosmos/Blob network access as required
- Keep secrets in Key Vault
- Avoid putting secrets in Git or notebooks
- Use least-privilege managed identities where possible

19.4 Prompt injection awareness

UC2 should treat uploaded finding text as untrusted content. System prompts must instruct the model to **use findings as data**, not as instructions that can override report policy.

---

20. Design Decisions and Trade-offs

Decision: Provider abstraction first

Benefit: same codebase for local and Azure.
Trade-off: more interfaces/boilerplate early.

Decision: Deterministic UC1, generative UC2

Benefit: reliability where it matters most (structure/validation), creativity where it helps (narrative).
Trade-off: UC2 quality depends on prompt/model tuning.

Decision: Simple sequential graphs

Benefit: easy to reason about and demo.
Trade-off: not yet a full durable workflow orchestrator.

Decision: Workspace-centric model

Benefit: natural engagement boundary for files/runs/audit.
Trade-off: cross-engagement analytics needs future design.

Decision: Human approval gate

Benefit: trust and compliance.
Trade-off: not fully autonomous; throughput depends on reviewers.

---

21. Glossary

Term	Meaning
TPRA	Third Party Risk Assessment
UC1	Use Case 1 — Structured Findings Package Agent
UC2	Use Case 2 — Draft Report Generation Agent
Provider	Swappable infrastructure adapter
Registry	Factory that selects providers from config
Workspace	One TPRA engagement container
Finding	Canonical normalized risk/issue record
Exception	Validation problem with a finding/field
Reviewer Log	Human decision records on findings
Foundry	Azure AI Foundry agent hosting/deployment concept
DaVinci	Client cloud target environment referenced by migration docs
HITL	Human-in-the-loop

---

22. Suggested NotebookLM Study Questions

Use these prompts after uploading this document:

1. Explain the full UC1 → human review → UC2 lifecycle in simple language.
2. What exactly changes when moving from local mock mode to Azure client mode?
3. Create a beginner onboarding checklist for a new developer on this repo.
4. Compare responsibilities of services vs repositories vs providers.
5. What outputs does UC1 produce and why does each output exist?
6. Where are prompts stored and why are they versioned?
7. Generate a quiz to test my understanding of the architecture layers.
8. What security controls are already implied by the design?
9. Draft a stakeholder briefing for a cybersecurity leader (non-technical).
10. Identify the top five extension points for the next MVP iteration.

---

23. Practical Mental Model (One Paragraph)

Think of this system as a **TPRA workbench**: a workspace holds the evidence for one vendor assessment; UC1 is the cleanup-and-packaging specialist; a human reviewer is the quality gate; UC2 is the report-writing assistant; and the provider registry is the power adapter that lets the same workbench plug into a laptop (local mocks) or the client's Azure cloud without rebuilding the tools.

---

24. Source Index for Further Reading in the Repo

- `README.md` — quickstart
- `docs/ARCHITECTURE.md` — concise architecture notes
- `docs/MIGRATION.md` — local-to-cloud migration steps
- `docs/Agent_TPRA_Overview.pdf` — original repository guide PDF
- `foundry/agents.yaml` — agent catalog source of truth
- `backend/config.yaml` — provider/config switchboard
- `backend/app/agents/structured_findings/graph.py` — UC1 implementation

- `backend/app/agents/draft_report/graph.py` — UC2 implementation
- `backend/app/providers/registry.py` — environment portability core
- `frontend/src/App.tsx` — operator UI workflow

---

25. Closing Notes for Learners

If you only remember five things, remember these:

1. **Two agents, one human gate:** UC1 structures, humans approve, UC2 drafts.
2. **Workspace is the container** for everything in an engagement.
3. **Providers make the app portable** between laptop and Azure.
4. **Domain models are the shared language** across API, services, and agents.
5. **Auditability and reviewability are features**, not afterthoughts.

This document is intentionally detailed so NotebookLM can answer both high-level stakeholder questions and deep engineering questions from the same source.